ÉTUDE DE CAS — STRIPE

LIVRABLE 8

Requêtes SQL et NoSQL

Certification RNCP 41993 – Adrien COSSON
Bloc 2 - Concevoir et déployer l'infrastructure de données et de calcul pour l'IA

Ce livrable présente un ensemble de requêtes qui montrent comment les modèles de données proposés permettent de répondre aux questions clés de Stripe : analyse des revenus, détection de fraude, segmentation client et suivi de la conformité. Les requêtes sont réparties selon la base utilisée :

- **Redshift (OLAP)** pour les analyses sur l'historique.
- **Aurora PostgreSQL (OLTP)** pour les besoins opérationnels en temps réel.
- **DocumentDB (NoSQL)** pour les données comportementales et le suivi du modèle.

1. Requêtes analytiques (Redshift – OLAP)

Requête 1 – Revenus mensuels par région

Question métier : *comment évoluent les revenus de Stripe chaque mois, selon la région des marchands ?*

```
SELECT d.year,
       d.month,
       m.region,
       SUM(a.total_amount_usd)  AS revenus_usd,
       SUM(a.transaction_count) AS nb_transactions
FROM agg_revenue_daily a
JOIN dim_date d     ON a.date_key = d.date_key
JOIN dim_merchant m ON a.merchant_key = m.merchant_key
WHERE d.year = 2026
GROUP BY d.year, d.month, m.region
ORDER BY d.month, revenus_usd DESC;
```

La requête utilise la table pré-agrégée agg_revenue_daily plutôt que la table de faits : elle ne lit qu'une ligne par marchand et par jour au lieu de millions de transactions.

Requête 2 – Top 10 des marchands sur les 90 derniers jours

Question métier : *quels marchands génèrent le plus de revenus, remboursements déduits ?*

```
SELECT m.name,
       m.region,
       SUM(a.total_amount_usd - a.refund_amount_usd) AS revenu_net_usd
FROM agg_revenue_daily a
JOIN dim_date d     ON a.date_key = d.date_key
JOIN dim_merchant m ON a.merchant_key = m.merchant_key
WHERE d.full_date >= DATEADD(day, -90, CURRENT_DATE)
GROUP BY m.name, m.region
ORDER BY revenu_net_usd DESC
LIMIT 10;
```

Requête 3 – Taux de fraude par pays sur le dernier mois

Question métier : *depuis quels pays proviennent le plus de fraudes ?*

```
SELECT c.name AS pays_ip,
       SUM(f.transaction_count) AS nb_transactions,
       SUM(f.fraud_count)       AS nb_fraudes,
       ROUND(100.0 * SUM(f.fraud_count) / NULLIF(SUM(f.transaction_count), 0), 2) AS taux_fraude_pct,
       SUM(f.fraud_amount_usd)  AS montant_fraude_usd
FROM agg_fraud_daily f
JOIN dim_country c ON f.ip_country_key = c.country_key
JOIN dim_date d    ON f.date_key = d.date_key
WHERE d.full_date >= DATEADD(month, -1, CURRENT_DATE)
GROUP BY c.name
HAVING SUM(f.transaction_count) >= 1000
ORDER BY taux_fraude_pct DESC
LIMIT 10;
```

Le filtre HAVING écarte les pays avec trop peu de transactions, pour lesquels un taux de fraude ne serait pas représentatif.

Requête 4 – Fraude selon l'appareil et l'heure

Question métier : *à quels moments et sur quels appareils la fraude est-elle la plus fréquente ?*

```
SELECT device_type,
       hour_of_day,
       COUNT(*) AS nb_paiements,
       SUM(CASE WHEN is_fraud THEN 1 ELSE 0 END) AS nb_fraudes,
       ROUND(100.0 * SUM(CASE WHEN is_fraud THEN 1 ELSE 0 END) / COUNT(*), 2) AS taux_fraude_pct
FROM fact_transaction
WHERE transaction_type = 'payment'
GROUP BY device_type, hour_of_day
ORDER BY taux_fraude_pct DESC;
```

Le résultat aide l'équipe fraude à comprendre les comportements frauduleux et peut suggérer de nouvelles features pour le modèle.

Requête 5 – Calcul des indicateurs RFM par client

Question métier : *quels sont la récence, la fréquence et le montant des achats de chaque client ?*

```
SELECT f.customer_key,
       DATEDIFF(day, MAX(d.full_date), CURRENT_DATE) AS recence_jours,
       COUNT(*)                                      AS frequence,
       SUM(f.amount_usd)                             AS montant_usd
FROM fact_transaction f
JOIN dim_date d ON f.date_key = d.date_key
WHERE f.transaction_type = 'payment'
  AND f.status = 'successful'
  AND d.full_date >= DATEADD(month, -12, CURRENT_DATE)
GROUP BY f.customer_key;
```

Ce résultat sert d'entrée au modèle de segmentation (K-means) décrit dans le livrable 7.

Requête 6 – Revenus par segment client

Question métier : *quels segments de clients rapportent le plus ?*

```
SELECT cu.segment,
       COUNT(DISTINCT f.customer_key) AS nb_clients,
       SUM(f.amount_usd)              AS revenus_usd,
       ROUND(SUM(f.amount_usd) / COUNT(DISTINCT f.customer_key), 2) AS revenu_moyen_par_client
FROM fact_transaction f
JOIN dim_customer cu ON f.customer_key = cu.customer_key
JOIN dim_date d      ON f.date_key = d.date_key
WHERE f.transaction_type = 'payment'
  AND f.status = 'successful'
  AND d.year = 2026
GROUP BY cu.segment
ORDER BY revenus_usd DESC;
```

Requête 7 – Produits les plus vendus sur les 30 derniers jours

Question métier : *quels produits fonctionnent le mieux ?*

```
SELECT p.name          AS produit,
       p.merchant_name AS marchand,
       SUM(a.quantity_sold) AS quantite_vendue,
       SUM(a.revenue_usd)   AS revenus_usd
FROM agg_product_daily a
JOIN dim_product p ON a.product_key = p.product_key
JOIN dim_date d    ON a.date_key = d.date_key
WHERE d.full_date >= DATEADD(day, -30, CURRENT_DATE)
GROUP BY p.name, p.merchant_name
ORDER BY revenus_usd DESC
LIMIT 20;
```

2. Requêtes opérationnelles (Aurora PostgreSQL – OLTP)

Requête 8 – Paiements d'un client sur la dernière heure

Question métier : *ce client a-t-il fait beaucoup de paiements en peu de temps ?*

```
SELECT COUNT(*)                AS nb_paiements_1h,
       SUM(transaction_amount) AS montant_total
FROM "transaction"
WHERE customer_id = 42
  AND transaction_type = 'payment'
  AND transaction_date_time >= NOW() - INTERVAL '1 hour';
```

Grâce à l'index (customer_id, transaction_date_time), la requête est très rapide. Elle sert à vérifier une alerte ; en production, cette feature est lue directement dans Redis.

Requête 9 – Paiements acceptés malgré un score de fraude élevé

Question métier : *quels paiements récents l'équipe fraude doit-elle vérifier en priorité ?*

```
SELECT t.transaction_id,
       t.customer_id,
       t.merchant_id,
       t.transaction_amount,
       t.currency_code,
       t.fraud_score,
       c.iso_code AS pays_ip
FROM "transaction" t
LEFT JOIN country c ON t.ip_country_id = c.country_id
WHERE t.status = 'successful'
  AND t.fraud_score >= 0.7
  AND t.transaction_date_time >= NOW() - INTERVAL '24 hours'
ORDER BY t.fraud_score DESC;
```

Requête 10 – Contrôle des remboursements

Question métier : *existe-t-il des paiements remboursés pour un montant supérieur au paiement d'origine ?*

```
SELECT p.transaction_id,
       p.transaction_amount      AS montant_initial,
       SUM(r.transaction_amount) AS montant_rembourse
FROM "transaction" p
JOIN "transaction" r
  ON r.parent_transaction_id = p.transaction_id
 AND r.transaction_type = 'refund'
GROUP BY p.transaction_id, p.transaction_amount
HAVING SUM(r.transaction_amount) > p.transaction_amount;
```

La requête utilise la relation de la table transaction vers elle-même (parent_transaction_id). Un résultat non vide signale une anomalie ou une tentative de fraude sur les remboursements.

Requête 11 – Historique des accès aux données d'un client (RGPD)

Question métier : *qui a consulté ou modifié les données de ce client ?*

```
SELECT occurred_at,
       actor,
       action,
       entity,
       entity_id
FROM audit_log
WHERE customer_id = 42
ORDER BY occurred_at DESC;
```

Cette requête permet de répondre à un client qui exerce son droit d'accès, ou à un auditeur (livrable 6).

3. Requêtes NoSQL (DocumentDB)

Les requêtes suivantes reposent sur trois collections :

- **sessions :** une session par visite, avec la liste de ses événements imbriqués ;
- **avis_clients :** un document par avis ;
- **fraud_predictions :** un document par prédiction du modèle de fraude.

Exemple de document pour chaque collection :

```
// sessions
{ "session_id": "S-4521", "customer_id": 42, "device": "mobile", "ip_country": "FR",
  "started_at": ISODate("2026-10-12T10:02:10Z"),
  "events": [
    { "ts": ISODate("2026-10-12T10:02:15Z"), "type": "page_view", "page": "/panier" },
    { "ts": ISODate("2026-10-12T10:02:41Z"), "type": "payment_attempt", "payment_method_id": 881 }
  ] }

// avis_clients
{ "review_id": 9001, "customer_id": 42, "merchant_id": 17, "rating": 2,
  "comment": "Paiement débité deux fois", "created_at": ISODate("2026-10-11T18:20:00Z") }

// fraud_predictions
{ "transaction_id": 5001, "customer_id": 42, "score": 0.83, "decision": "refused",
  "model_version": "fraud-v12", "predicted_at": ISODate("2026-10-12T10:02:42Z") }
```

Requête 12 – Sessions avec de nombreuses cartes testées

Question métier : *quelles sessions ressemblent à un test de cartes volées ?*

```
db.sessions.aggregate([
  { $match: { started_at: { $gte: ISODate("2026-10-11T00:00:00Z") } } },
  { $unwind: "$events" },
  { $match: { "events.type": "payment_attempt" } },
  { $group: {
      _id: "$session_id",
      customer_id: { $first: "$customer_id" },
      cartes: { $addToSet: "$events.payment_method_id" }
  } },
  { $project: { customer_id: 1, nb_cartes: { $size: "$cartes" } } },
  { $match: { nb_cartes: { $gt: 3 } } },
  { $sort: { nb_cartes: -1 } }
]);
```

Les événements étant imbriqués dans la session, on récupère tout le parcours en une seule lecture. Un index sur started_at limite la recherche aux sessions récentes.

Requête 13 – Marchands les moins bien notés

Question métier : *quels marchands ont les plus mauvais avis clients ?*

```
db.avis_clients.aggregate([
  { $match: { created_at: { $gte: ISODate("2026-07-01T00:00:00Z") } } },
  { $group: {
      _id: "$merchant_id",
      note_moyenne: { $avg: "$rating" },
      nb_avis: { $sum: 1 }
  } },
  { $match: { nb_avis: { $gte: 50 } } },
  { $sort: { note_moyenne: 1 } },
  { $limit: 10 }
]);
```

Requête 14 – Avis négatifs mentionnant un problème de paiement

Question métier : *quels clients signalent une fraude ou un double prélèvement ?*

```
db.avis_clients.find(
  { rating: { $lte: 2 },
    comment: { $regex: "fraude|arnaque|deux fois|non autorisé", $options: "i" } },
  { customer_id: 1, merchant_id: 1, comment: 1, created_at: 1 }
).sort({ created_at: -1 });
```

Requête 15 – Suivi du modèle de fraude

Question métier : *le comportement du modèle change-t-il d'un jour à l'autre ou d'une version à l'autre ?*

```
db.fraud_predictions.aggregate([
  { $match: { predicted_at: { $gte: ISODate("2026-10-05T00:00:00Z") } } },
  { $group: {
      _id: {
        version: "$model_version",
        jour: { $dateToString: { format: "%Y-%m-%d", date: "$predicted_at" } }
      },
      nb_predictions: { $sum: 1 },
      score_moyen: { $avg: "$score" },
      nb_refus: { $sum: { $cond: [ { $eq: ["$decision", "refused"] }, 1, 0 ] } }
  } },
  { $sort: { "_id.jour": 1 } }
]);
```

Une hausse brutale du score moyen ou du nombre de refus peut signaler une dérive du modèle (livrable 7).

4. Requête de conformité (Redshift – OLAP)

Requête 16 – Accès aux données clients par acteur et par semaine

Question métier : *qui consulte ou modifie les données des clients, et ce volume est-il normal ?*

```
SELECT d.year,
       d.week,
       a.actor,
       a.action,
       COUNT(*)                        AS nb_actions,
       COUNT(DISTINCT a.customer_key) AS nb_clients_concernes
FROM fact_audit a
JOIN dim_date d ON a.date_key = d.date_key
WHERE a.entity = 'customer'
  AND d.full_date >= DATEADD(week, -4, CURRENT_DATE)
GROUP BY d.year, d.week, a.actor, a.action
ORDER BY d.year, d.week, nb_actions DESC;
```

Cette requête alimente un rapport de conformité produit automatiquement chaque semaine (livrable 6). Un acteur qui consulte beaucoup plus de fiches clients que d'habitude peut signaler un accès abusif. Elle utilise la table fact_audit, alimentée à partir de la table audit_log de l'OLTP (livrable 3).

5. Synthèse

Question métier	Requêtes	Base
Analyse des revenus	1, 2	Redshift
Performance des marchands et des produits	2, 7, 13	Redshift, DocumentDB
Détection de fraude	3, 4, 8, 9, 10, 12, 14, 15	Redshift, Aurora, DocumentDB
Segmentation client	5, 6	Redshift
Conformité	11, 16	Aurora, Redshift